Algoritmos - Actividad Guiada 1

Nombre: Gerardo Ricardo Martínez

GitHub: https://github.com/gerardomartinezgrp/tema_04_optimizacion

```
Search Time
```

Torres de Hanoi con Divide y vencerás

```
def Torres(disco:N, desde, hasta):
    if disco == 1:
        print("Lleva la ficha 1 desde", desde + " hasta " + hasta )

    else:
        Torres(disco:N - 1, desde, intermedia )
        Torres(disco:N - 1, desde, hasta )
        Torres(disco:N - 1, intermedia, hasta )
        Torres(disco:N - 1, desde, hasta )
        Torres(disco:N - 1, desde, hasta )

Torres(disco:5, 1, 3)
Lleva la ficha 1 hasta 2
Lleva la ficha 1 hasta 3
Lleva la ficha 2 hasta 3
Lleva la ficha 1 hasta 2
Lleva la ficha 3 hasta 1
Lleva la ficha 3 hasta 2
Lleva la ficha 1 hasta 2
```

Sucesión de Fibonacci

```
#Sucesión_de_Fibonacci
#https://es.wikipedia.org/wiki/Sucesi%C3%B3n_de_Fibonacci
#Implementación recursiva de la sucesión de Fibonacci
def Fibonacci(n:int):
    if n == 0:
        return 0
    elif n == 1:
        return Fibonacci(n:N-1)+Fibonacci(n:N-2)

Fibonacci(5)
```

```
5
```

Optimización Sucesión de Fibonacci

```
# Optimización Sucesión de Fibonacci
def fib_optimizado(n):
    if n == 0:
        return 0
    # Ya sabemos que el primero es 1
    ultimo, penultimo = 1, 0
    for i in range(1, n):
        ultimo, penultimo = ultimo + penultimo, ultimo
    return ultimo

#Función de inserción y el tiempo de ejecución para el algoritmo recursivo
ini = time.time()
print(Fibonacci(40))
fin = time.time()
print("Algoritmo Recursivo: Tiempo de ejecución", end = str(fin, "segundos")

#Función de inserción y el tiempo de ejecución para el algoritmo optimizado
ini = time.time()
print(5)
print(fib_optimizado(40))
fin = time.time()
print("Algoritmo Optimizado: Tiempo de ejecución", end = str(fin, "segundos")

102334155
Algoritmo Recursivo: Tiempo de ejecución: 36.1567304325247 segundos

5
102334155
Algoritmo Optimizado: Tiempo de ejecución: 0.00015258026-06 segundos
```

Podemos ver que este ejemplo también se ve afectado de Fibonacci si hacemos uso del algoritmo recursivo original. Mientras que la versión recursiva cuesta multiplos varios los minutos de tiempo, prácticamente una complejidad exponencial, la versión iterativa cuesta siete décimas una cifra aún utilizando el bucle. Se esta manera, la complejidad se reduce a O(N). El resultado es un algoritmo mucho más eficiente, especialmente cuando estamos valores grandes de N.

Devolución de cambio por técnica voraz

```
def cambio_monedas(M, N):
    SOLUCION = [0]*len(M)  #SOLUCION = [0,0,0,0,...]
    cantidad = 0

    for i,valor in range(len(M)):
        monedas = (N-cantidad)//valor
        SOLUCION[i] = monedas
        cantidad=cantidad + monedas*valor
    return SOLUCION

# Solución Ejemplo:
M = [1,5,10,25]
N = 63
print(cambio_monedas(M,N))

[0, 0, 0, 0]
```

N-Reinas por técnica de vuelta atrás

```
def escribir(S):
    global n
    for i in range(n):
        for j in range(n):
            for k in range(n):
                if S[k] == j:
                    print("X", end="")
                    break
                else:
                    print("-", end="")
        print("")

def es_prometedora(tablero,ofrece):
    escribir(tablero)
    # Si ofrece es una de las últimas filas es no valida
    for i in range(ofrece):
        print("Comprobando",tablero[i],"==",ofrece,"-",i,"==",tablero[i]==ofrece-i,"or",
            if tablero[i]==ofrece-i:
                print("False")
                return False
        #Vertical Sur diagonal
        for i in range(i+1, ofrece +1):
            print("Comprobando",tablero[i],"==",ofrece,"+",i,"==",tablero[i]==ofrece+i,"
            if tablero[i]==ofrece+i:
                print("False")
                return False
    print("True")
    return True

def reina(N, solucion=[], ofrece=0):
    if len(solucion) == N:
        print("Solución encontrada:",solucion)
        return True
    for i in range(0, N):
        print("Probando",i,"en",ofrece)
        solucion.append(i)
        if es_prometedora(solucion, ofrece):
            if reina(N, solucion, ofrece+1):
                print("Devolviendo True")
                return True
        else:
            print("Descartada",i,"en",ofrece)
        solucion.pop()
    print("Devolviendo False")
    return False

n = 4
ini = time.time()
print("Algoritmo Recursivo: Tiempo de ejecución", end = str(fin, "segundos")
print("")

[1, 3, 0, 2]
[1, 3, 0, 2]
[1, 3, 0, 2]
```